Candidate Assessment Portal
Timed take-home assignments for hiring teams

Ajaia LLC - AI-Native Full Stack Developer Assignment

AI-Native Full Stack Developer Assignment

Candidate: Aniruddh Laharia (aniruddh.laharia@gmail.com)

Time remaining: 239:42

AI-Native Full Stack Developer Assignment

Objective

This assignment is designed to test how you handle a realistic product-engineering problem with ambiguous scope, multiple surfaces, and tight delivery constraints. We want to see how you prioritize, what you build, how you communicate tradeoffs, and how you use AI tools without outsourcing judgment.

Time Limit

Please spend no more than 4–6 hours on this assignment.

If you reach the time limit, stop and submit what you have. A focused, well-reasoned partial solution is better than an overextended build.

Scenario

Ajaia is exploring internal productivity tools that help teams move faster on shared work. For this exercise, assume you have been asked to build a lightweight collaborative document editor inspired by Google Docs.

The goal is not to recreate Google Docs completely. The goal is to ship the strongest working version you can within the timebox while showing sound product judgment, full stack capability, and clear prioritization.

Your app should demonstrate how you think about document creation, editing, file handling, sharing, and usability in a real product environment.

Tasks

Build a small full stack application that includes the following core capabilities:

1. Document Creation and Editing

Users should be able to:

- Create a new document
- Rename a document
- Edit document content in a browser
- Save and reopen documents

The editing experience should support basic rich-text formatting such as:

- Bold
- Italic
- Underline
- Headings or text size variation
- Bulleted or numbered lists

You do not need to match Google Docs exactly, but the editing flow should feel usable and coherent.

2. File Upload

Allow a user to upload at least one file into the product workflow. You may choose the exact behavior, but it should be product-relevant. Examples include:

- Uploading a `.txt`, `.md`, or `.docx` file and turning it into a new editable document
- Uploading an attachment that is associated with a document
- Importing content from a file into an existing draft

If you limit supported file types, state that clearly in the UI and README.

3. Sharing

Implement a simple sharing model so that one user can share a document with another. This does not need to be enterprise-grade access control, but it should demonstrate clear intent and working logic.

At minimum, include:

- A document owner
- A way to grant another user access
- A visible distinction between owned and shared documents

You may simulate users with seeded accounts, mocked auth, or a lightweight login flow if that keeps the scope reasonable.

4. Persistence

Persist documents and sharing data so that:

- Documents remain available after refresh
- Formatting or structure is preserved in a reasonable way
- Shared access behavior can be demonstrated

You may use any practical storage approach for this scope, including SQLite, Postgres, Supabase, or a local file-based store if well documented.

5. Product and Engineering Quality

Include enough engineering quality to show how you work in practice. At minimum, include:

- Clear setup and run instructions
- A working deployment reviewers can access via your preferred deployment path
- Basic validation and error handling
- At least one meaningful automated test
- A short architecture note explaining what you prioritized and why

AI-Native Workflow Note

Because this is an AI-forward role, include a short note explaining:

- Which AI tools you used
- Where AI materially sped up your work
- What AI-generated output you changed or rejected
- How you verified correctness, UX quality, and implementation reliability

We are evaluating practical AI usage, not volume of AI usage.

Walkthrough Video

Record a 3–5 minute walkthrough video that covers:

- The main user flow
- What functionality works end to end
- What you intentionally deprioritized
- Key implementation decisions
- How AI supported your workflow

An unlisted Loom or YouTube link is fine.

Deliverables

Submit one Google Drive folder containing:

- The source code
- A `README.md` with local setup and run instructions
- A short architecture note in Markdown or PDF
- Your AI workflow note in Markdown or PDF
- A `SUBMISSION.md` file listing exactly what is included
- A live product URL we can test, deployed via your preferred path
- A text file with the walkthrough video URL
- Screenshots or a short demo GIF if setup requires extra steps

Submission Format

Your submission should make it easy for reviewers to evaluate quickly. Please include:

- One Google Drive folder link containing all materials
- A live deployment link that we can use to test the product
- Any credentials, seeded users, or test accounts needed to review sharing flows
- Clear instructions for how to run the project locally

If any feature is partial or incomplete, state:

- What is working
- What is incomplete
- What you would build next with another 2–4 hours

Constraints

- Keep the project intentionally scoped
- Do not try to build every Google Docs feature
- Prioritize depth in a few important areas over shallow coverage everywhere
- You may use any language, framework, editor library, or tooling stack
- You may use AI coding tools and assistants
- Do not require reviewers to pay for a dependency or service

We are evaluating product judgment as much as implementation skill. Strong candidates usually make deliberate scope cuts and explain them clearly.

What We Will Evaluate

- Ability to turn an open-ended prompt into a coherent product slice
- Full stack execution across frontend, backend, persistence, and access logic
- Quality of the document editing experience within the chosen scope
- Practical handling of file upload and sharing behavior
- Basic infrastructure and deployment judgment, including ability to ship a testable build
- Code clarity, maintainability, and delivery discipline
- Prioritization and tradeoff awareness under time pressure
- Written and verbal communication quality
- Mature use of AI tools without sacrificing engineering standards

Optional Stretch

If you finish early, you may add one small enhancement such as:

- Real-time collaboration indicators
- Commenting or suggestion mode
- Document version history
- Export to PDF or Markdown
- Role-based sharing permissions beyond basic access

This is optional. Do not sacrifice core functionality to pursue stretch work.

Video Submission Link

https://youtu.be/... or other public video link

Required. Paste a public Loom, YouTube, Vimeo, Google Drive, or similar video link.

Submission (Markdown, max 1000 lines)

Submit Assignment

Saved at 2:27:56 PM

If time runs out, your latest autosaved content is submitted automatically.